Neural Caption-Image Retrieval

UE24CS352A Machine Learning Mini-Project, Problem 36 (Image-caption retrieval)
Tatam Mahalakshmi (PES2UG24CS558), T S Sathya Kiran (PES2UG24CS547)

Problem statement

Given a sentence describing a picture, find the matching pictures in a collection of images. The input is the collection of images and a text query, and the output is a ranked list of the most relevant images. We test it by taking each caption, ranking all the images in the collection, and checking how high the image that caption was written for appears.

Dataset

We use the Microsoft COCO dataset, where every image has 5 captions written by people. We trained on 40,000 random images from the COCO train set (200,000 image-caption pairs). For validation we used 500 images and for testing 1,000 images, taken from the COCO validation set, so no image appears in more than one split. The test set has 5,000 queries (5 captions for each of the 1,000 images). The original paper used all of COCO (about 123,000 images); we used a smaller part because of the limited compute on Colab.

Approach

The idea is to turn images and sentences into lists of numbers in the same space, so that a sentence ends up close to its own image. To search, we convert the query to numbers and return the nearest images.

Image side. We use a pretrained VGG-19 network and take the 4096 numbers from its second-to-last layer. This network is not trained by us. A single linear layer then reduces these to 1024 numbers.

Text side. Each word is converted to a 300-number GloVe vector (pretrained). A GRU (or LSTM) reads the sentence word by word, and its final state (1024 numbers) represents the whole sentence. The GloVe vectors are fine-tuned during training.

Training. We use a ranking loss. In each batch of 128 pairs, a caption should be closer to its own image than to the other images, and an image should be closer to its own caption than to the other captions, by some margin. We tried two ways of measuring closeness: normal distance, and the order-embedding method from Vendrov et al.

Baseline. For comparison we average the GloVe vectors of the words in a caption and use ridge regression to predict the VGG features of the image from that average. This ignores word order.

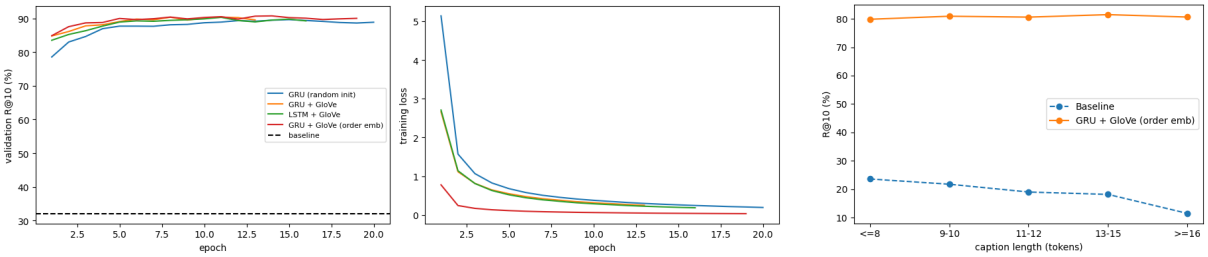
Implementation

Everything is written in PyTorch and run in Google Colab on a T4 GPU. We extract the VGG features once and save them, then train on the saved features. Training uses the Adam optimizer (learning rate 0.0002), batch size 128 and dropout 0.2, for at most 25 epochs, stopping early if the validation score does not improve for 5 epochs. The best model was chosen using the validation set, and the test set was used only for the final numbers. We compared four neural models: GRU with random word vectors, GRU with GloVe, LSTM with GloVe, and GRU with GloVe using order embedding. We also wrote a separate demo notebook that loads the trained model and lets us type any query and see the results next to the baseline's.

Results

R@K is the percentage of test queries where the correct image is in the top K results out of 1,000. Higher is better for R@K, lower is better for rank.

Method	R@1	R@5	R@10	Median rank	Mean rank
Baseline (ridge regression)	4.4	13.7	20.6	82	190.1
GRU, random word vectors	30.3	64.6	79.5	3	11.4
GRU + GloVe	29.5	65.1	80.4	3	10.2
LSTM + GloVe	31.1	66.2	80.3	3	10.2
GRU + GloVe, order embedding	32.3	67.1	80.7	3	9.7



Left: validation R@10 and training loss for each epoch (dashed line is the baseline). Right: test R@10 for short and long captions.

Conclusions

The neural models are much better than the baseline. The best one (GRU + GloVe with order embedding) reached 80.7% R@10 and 32.3% R@1, compared to 20.6% and 4.4% for the baseline, and the median rank of the correct image went from 82 to 3.

The four neural models are close to each other (79.5% to 80.7% R@10) and we trained each only once, so we cannot say that one is clearly better. GloVe helped early in training but only improved the final R@10 by 0.9 points over random word vectors.

The caption length plot shows why the neural model helps. For the baseline, accuracy drops as captions get longer (about 24% for short captions to about 11% for long ones), because averaging hides the important words. The neural model stays around 80% for all lengths.

Our model works well on common scenes and objects but is weaker on relations like "in front of" and on words that are rare in the training captions. Our result for GRU + GloVe (80.4% R@10) is a bit lower than the paper's 86.8%, which is expected since we used about a third of the training images. Possible improvements are training on the full dataset, using attention over parts of the image, and fine-tuning the image network.